

# Case File Management & Analytics Portal (Legal Services)

Team : 11

|            |            |
|------------|------------|
| P.Ruhitha  | 2520080053 |
| M.Ananya   | 2520080069 |
| S.Himasree | 2520090181 |

# Problem Statement

Legal case information is often maintained across multiple files, documents, and manual records, making it difficult to efficiently store, retrieve, update, and track case details. This can result in data duplication, inconsistent records, difficulty in monitoring hearings and case status, and delays in accessing important information. There is a need for a centralized database system that organizes legal case records and enables efficient retrieval, tracking, and analysis of case-related information.

# Abstract

Legal case management involves handling a large amount of information, such as case details, clients, lawyers, judges, hearings, documents, and judgments. When these records are managed manually or stored across different files, it can become difficult to find information, track case progress, monitor hearings, and understand the workload of legal professionals. The proposed Case File Management & Analytics Portal is a centralized web application designed to make legal case management simpler and more organized. The system will store case-related information in a structured MySQL database, allowing users to easily add, update, search, and retrieve records. By connecting cases with their clients, lawyers, judges, hearings, documents, and judgments, the system provides a clear view of the complete history and current status of each case.

# Abstract

The portal will also include SQL-based analytics to provide useful insights, such as pending and resolved cases, case distribution by type and status, active cases handled by lawyers, hearing statistics, document counts, and average case duration. These results will be presented through summary tables and dashboard metrics to make the information easier to understand. Overall, the project aims to improve the organization, consistency, accessibility, and retrieval of legal information while demonstrating key Database Management System concepts such as relational schema design, normalization, primary and foreign keys, constraints, joins, filtering, grouping, and aggregate queries. The system will be developed using React for the frontend, Node.js for the backend, and MySQL for database management.

# Objectives

- To develop a centralized system for storing and managing legal case information.
- To reduce data duplication and inconsistencies caused by maintaining records across multiple files.
- To provide easy access to case details, clients, lawyers, judges, hearings, documents, and judgments.
- To enable efficient searching, updating, and retrieval of case information.
- To track case status, hearing schedules, and case progress.
- To generate useful analytics such as pending/resolved cases, case distribution, lawyer workload, and hearing statistics.
- To demonstrate DBMS concepts such as normalization, primary keys, foreign keys, constraints, joins, grouping, and aggregate queries.

# Proposed System

## Main Features:

- Case Management – Add, update, search, and track legal cases.
- Client Management – Maintain client information and link clients with their cases.
- Lawyer Management – Store lawyer details and track their active cases.
- Judge Management – Maintain information about judges handling cases.
- Hearing Management – Record hearing dates, outcomes, and next hearing dates.
- Document Management – Maintain case-related document records.
- Judgment Management – Store judgment details and final decisions.
- Analytics Dashboard – Display statistics about case status, types, hearings, workload, and duration.

# Methodology

## Requirement Analysis

Identify the information that needs to be managed, including cases, clients, lawyers, judges, hearings, documents, and judgments.

## Frontend Development

Develop the user interface using React, HTML, CSS, and JavaScript. The interface will allow users to add, update, search, and retrieve case information.

## Backend Development

Use Node.js to handle communication between the frontend and MySQL database and process user requests.

## SQL-Based Analytics

Create SQL queries using joins, filtering, grouping, and aggregate functions to generate information such as pending/resolved cases, lawyer workloads, hearing statistics, document counts, and average case duration.

## Dashboard & Testing

Present the analytical results through summary tables and dashboard metrics, and test the system to ensure that data can be correctly added, updated, retrieved, and analyzed.

# Frontend Design

## Role-Based Dashboards

Separate interfaces for Lawyer and Client users.

## Easy Navigation

Sidebar navigation for cases, hearings, documents and analytics.

## Case Management

Search, filter and track cases using clear status indicators.

## Hearing Tracking

Upcoming hearings, dates, courts and case timelines.

## Data Visualization

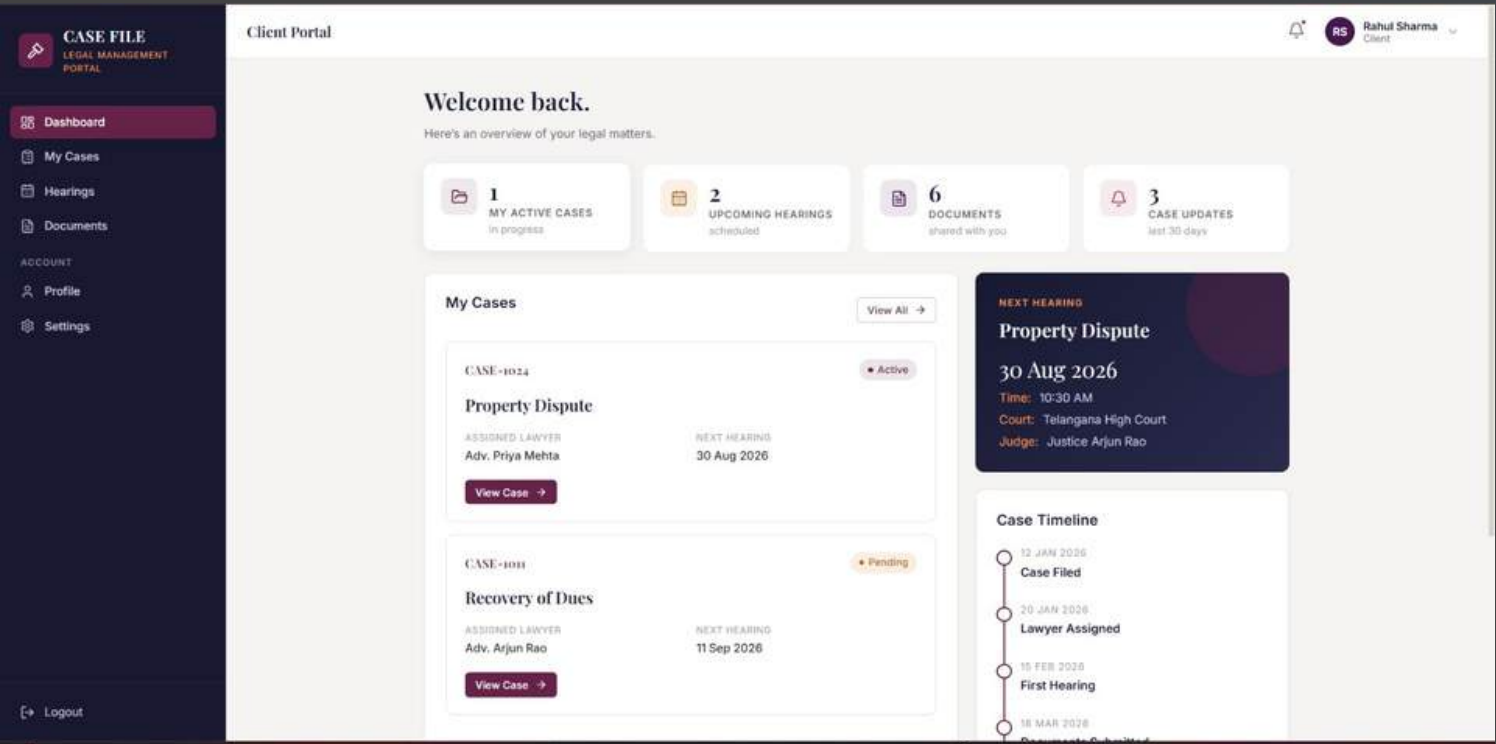
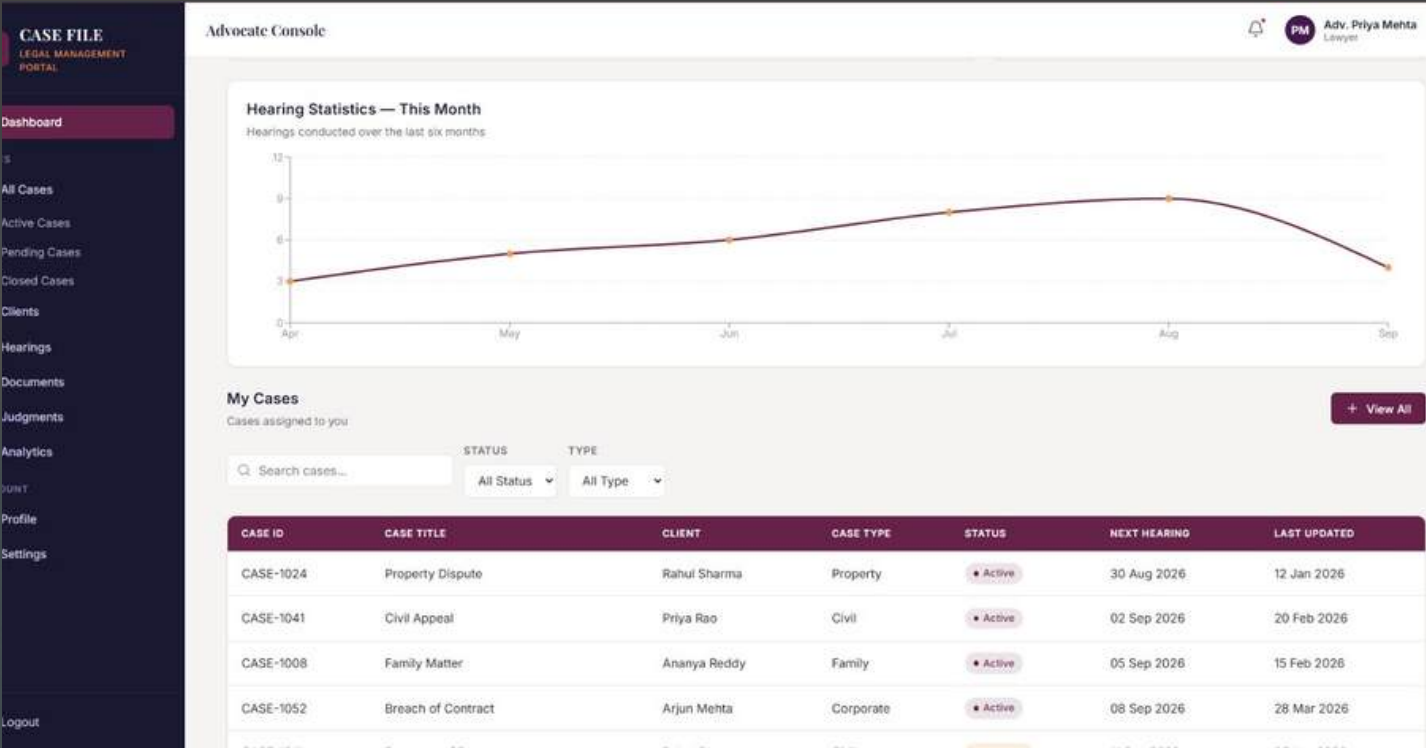
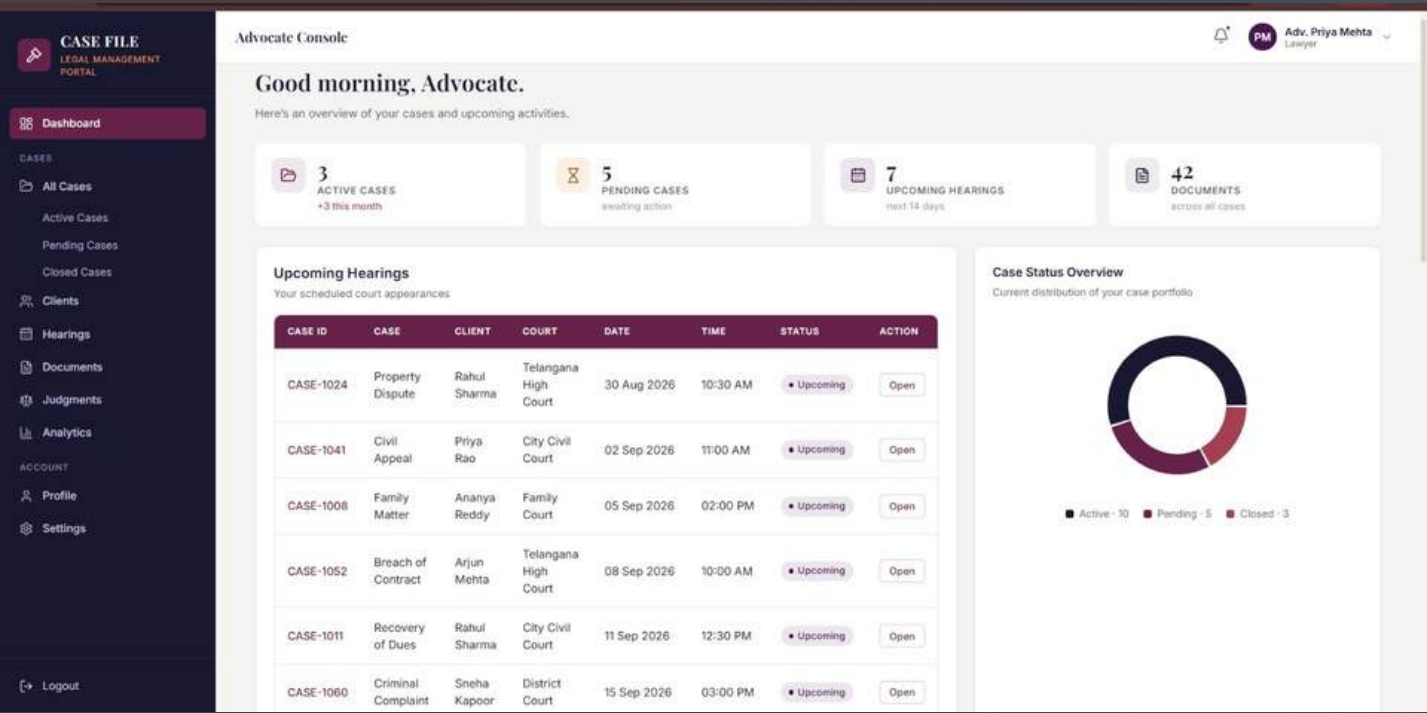
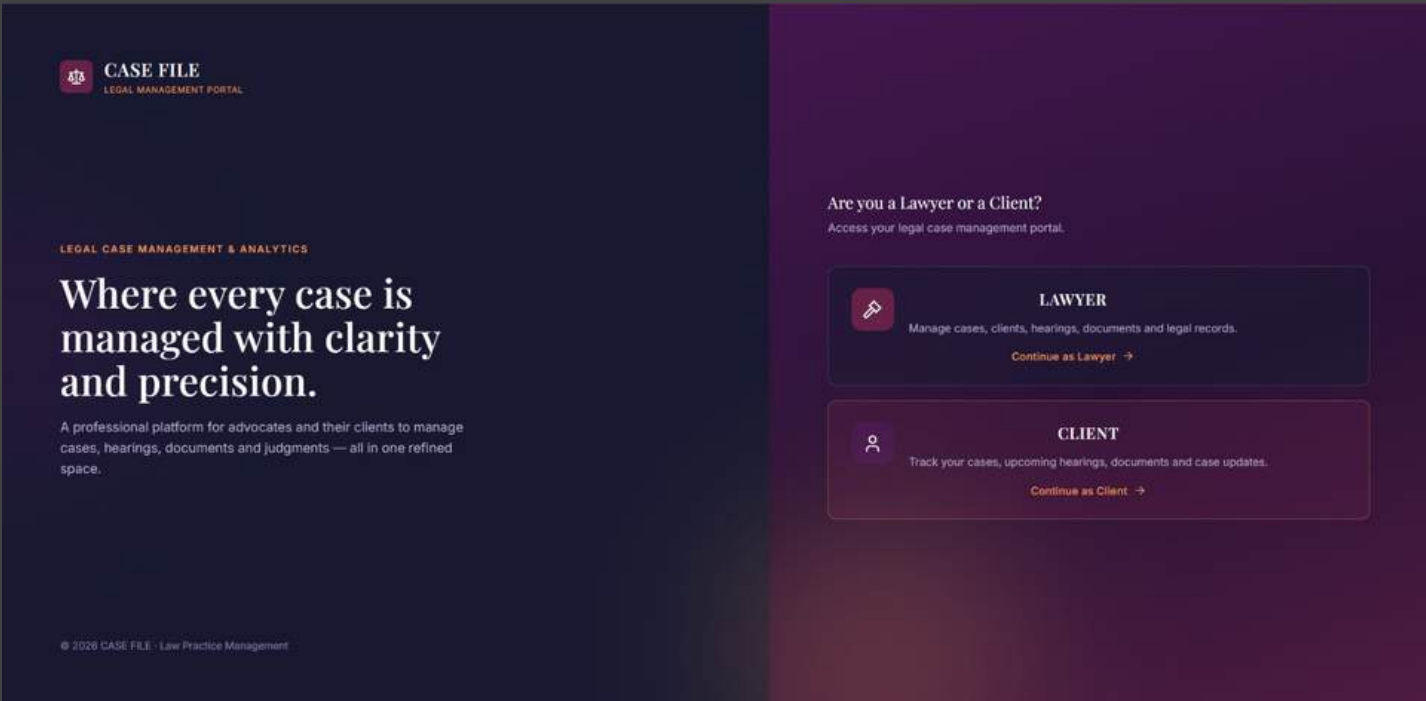
Charts and dashboard cards present key case statistics.

## User-friendly Interface

Buttons, forms, search bars and controls improve usability.



# Frontend Implementation



# Database Design

## user

PK user\_id • name • email  
password • role • phone  
UNIQUE: email

## clients

PK client\_id • FK user\_id •  
name • email password •  
created\_at • phone  
UNIQUE: email

## lawyers

PK lawyer\_id • FK user\_id  
bar\_number • specialization  
experience\_years • UNIQUE  
bar no

## judges

PK judge\_id • name  
court • specialization  
contact details

## cases - central table

PK case\_id • UNIQUE case\_number  
title • case\_type • description  
filing\_date • status • court  
FK client\_id → Clients  
FK lawyer\_id → Lawyers

## Hearings

PK hearing\_id  
FK case\_id • FK judge\_id  
hearing\_date • hearing\_time  
court\_room • purpose •  
status

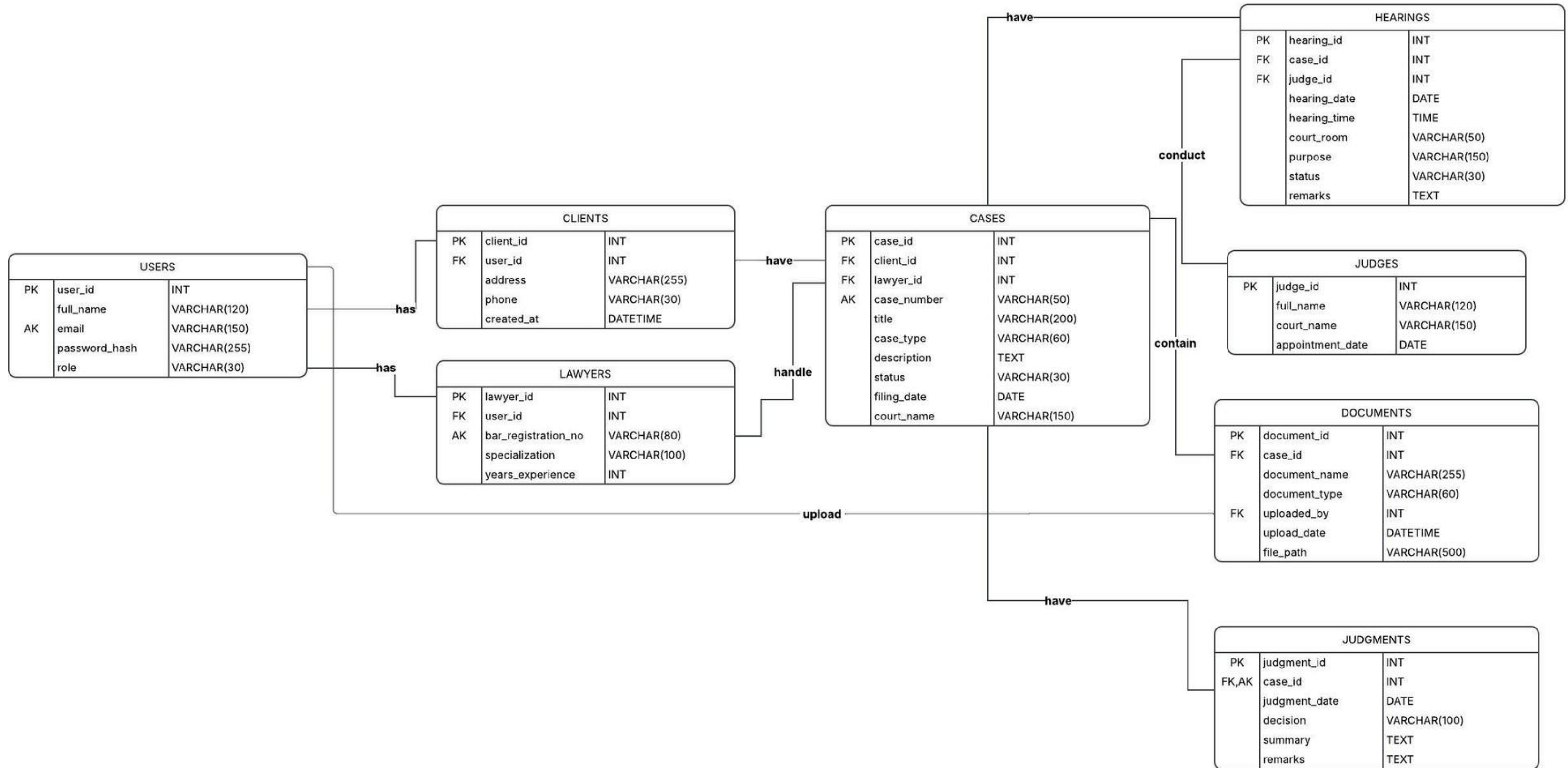
## Documents

PK document\_id  
FK case\_id • FK  
uploaded\_by  
document\_name • type  
upload\_date • file\_path

## judgements

PK judgment\_id  
FK + UNIQUE case\_id  
judgment\_date • decision  
summary • remarks  
1 Case → max 1 Judgment

# Database ER diagram



# 1. Software Used

- React + Vite
- FastAPI (Python)
- Spring Boot (Java) Microservices
- MySQL 8
- REST APIs
- Git/GitHub

# Challenges & Issues Faced

- Designing relationships between multiple legal entities.
- Avoiding duplicate data while storing information about cases, clients, lawyers, and hearings.
- Maintaining consistency between related tables using foreign keys.
- Handling different case statuses such as Active, Pending, and Resolved.
- Writing SQL queries involving multiple tables and joins.
- Generating meaningful analytics using GROUP BY and aggregate functions.
- Ensuring that updating or deleting records does not break relationships.
- Connecting the React frontend with the Node.js backend and MySQL database.
- Managing and retrieving large amounts of case-related information efficiently.

# Literature Survey

## 1. eCourts Services – Government of India

- Provides case status, cause lists, court orders and judgments.
- Cases can be searched using CNR, case number, party name, advocate name, FIR number, case type, etc.
- Provides a case history showing events and orders associated with a case.
- NJDG provides large-scale judicial statistics and analytical information.

## 2. Clio Manage

- Provides centralized case, client, document and calendar management.
- Supports case-status tracking, document management, task management and billing.
- Provides customizable reports and dashboards for law-firm performance.
- Focuses on complete law-firm practice management, rather than only case records.

# Literature Survey

## 3. MyCase

- Provides case management, document management and legal calendaring.
- Supports automated workflows, task management and client communication.
- Provides case analytics and customizable reports for caseloads, case status and firm performance.
- Designed primarily as a comprehensive commercial practice-management platform.

## 4. Indian Legal-Tech Platforms

- Platforms such as Provakil provide case tracking, court information, litigation workflows and analytics for Indian legal professionals.
- Modern platforms increasingly combine case management with court-data integration, alerts and AI-based legal features

# Future Work

- Add user authentication and role-based access for administrators, lawyers, and other authorized users.
- Add automated hearing reminders and notifications.
- Allow secure uploading and downloading of case documents.
- Add advanced dashboard visualizations and reports.
- Implement advanced search and filtering.
- Develop a mobile-friendly version of the portal.

# Conclusion

The Case File Management & Analytics Portal provides a centralized and structured solution for managing legal case information. By connecting cases with clients, lawyers, judges, hearings, documents, and judgments, the system makes legal records easier to store, retrieve, update, and track.

The use of MySQL provides a reliable relational database structure with primary keys, foreign keys, constraints, and relationships. SQL-based analytics can provide useful insights into case status, lawyer workload, hearings, document counts, and case duration.

Overall, the proposed system reduces the limitations of manual and scattered record management while demonstrating important Database Management System concepts such as relational schema design, normalization, joins, filtering, grouping, and aggregate queries.

# THANK YOU